HU-02 · Matriz de estados de cada control

> Como Analista de Riesgos quiero clasificar cada control evaluado dentro de una matriz de estados explicitados, para reflejar con precisión el grado real de implementación operativa y legal de cada control.

Estado previo (revisión del 15-oct): C3 cumplido. C1 dependía de una decisión del PO (la API aceptaba 4 estados y `requiere_revision` solo la asignaba el sistema). C2 no se cumplía: la API aceptaba justificaciones vacías en todos los estados.

Criterio	Estado	Evidencia
C1 · 1 de 5 estados		`EstadoClasificacion.declarables()` = 5; la API acepta y persiste los 5
C2 · Justificación obligatoria en Parcial y Requiere Revisión		422 `JUSTIFICACION_OBLIGATORIA` en `POST` y `PUT`
C3 · Registro persistente	(ya cumplía)	Prueba de API que verifica cada estado en la tabla `respuestas`

1. Decisión del PO: "Requiere revisión" seleccionable y resuelta por el LLM

Decisión del 16-oct: **el usuario selecciona "Requiere revisión"** cuando no sabe cómo clasificar un control, y **el LLM decide** si es `cumplido`, `parcialmente_cumplido` o `no_cumplido`.

Flujo:

1. `POST /api/respuestas/` con `estado_clasificacion = "requiere_revision"` y justificación (obligatoria, ver sección 2). Queda `estado_clasificacion_final = requiere_revision`, `fuente = usuario`, sin brecha.
2. El usuario sube evidencia (`POST /api/respuestas/{id}/evidencias`): ahora se admite en `requiere_revision`, igual que en `cumplido` y `parcialmente_cumplido`.
3. El worker la valida con el LLM. El prompt `validar_evidencia/1.2.0` agrega la **regla 8**: si el usuario declaró `requiere_revision`, no propuso clasificación; el LLM debe elegir `cumplido` / `parcialmente_cumplido` / `no_cumplido` con el mismo rigor (la justificación es contexto, no prueba). El schema de salida ya solo permitía esas 3 clasificaciones.
4. Se aplican las mismas reglas de decisión de siempre (`_apply_classification_decision`, sin cambios):

Resultado del LLM	Estado final	Fuente	Brecha
Confianza ≥ umbral	Lo que decidió el LLM	`llm`	Si decide `no_cumplido`
Confianza < umbral	Sigue en `requiere_revision` (0 puntos)	`usuario`	No
Inyección detectada	`requiere_revision`	`seguridad`	No
Error de la capa de IA	Sin cambios	—	—

5. Lo que el LLM no resuelve lo cierra el Oficial de Cumplimiento con `PATCH /api/respuestas/{id}/revision`, que ahora acepta solo estados **resolutivos** (`cumplido`, `parcialmente_cumplido`, `no_cumplido`, `no_aplica`): "resolver" un control como `requiere_revision` no tiene sentido y da 400.

Se conserva siempre lo que declaró el usuario (`estado_clasificacion = requiere_revision`) y por separado el estado vigente (`estado_clasificacion_final`), para auditoría.

Al completar la evaluación (`POST /completar`): todo control declarado `requiere_revision` debe tener al menos una evidencia; si no, 409 `REVISION_SIN_EVIDENCIA` con los códigos. Sin evidencia el LLM no puede decidir y el control quedaría en 0 puntos sin que nadie lo revise. Si tiene evidencia pero el LLM no la resolvió, se permite completar y cuenta 0 (en el denominador), como ya era la regla para `requiere_revision`.

2. Justificación obligatoria

En `parcialmente_cumplido` y `requiere_revision`:

- Se normaliza: sin espacios en los extremos y con los espacios, tabs y saltos de línea repetidos colapsados a uno.
- Debe tener al menos **`JUSTIFICACION_MIN_CARACTERES` = 20** caracteres (configurable en `.env`).
- Debe contener letras: `"....................."` o `"12345678901234567890"` no valen.
- Si no cumple: **422** `{"codigo": "JUSTIFICACION_OBLIGATORIA", "mensaje": "La justificación es obligatoria para 'parcialmente cumplido': describe en al menos 20 caracteres … (se recibieron 11)."}`

Aplica igual en `POST` (crear o actualizar) y `PUT`, antes de tocar la BD. `cumplido`, `no_cumplido` y `no_aplica` siguen aceptando justificación vacía. Se guarda el texto tal como lo escribió el usuario. Las respuestas ya guardadas no se modifican: la regla aplica cuando se crean o cambian.

Archivos

Nuevos

- `tests/test_hu02_matriz_estados.py` (18 pruebas → 66 casos con parámetros)
- `CAMBIOS_HU02.md`

Modificados

- `app/domain/models/respuesta.py`: `declarables()` con 5 estados; `requiere_evidencia()` incluye `requiere_revision`; nuevos `requiere_justificacion()` y `resolutivos()`.
- `app/application/use_cases/create_respuesta.py`: `validar_justificacion()`, `normalizar_justificacion()` y `JustificacionObligatoriaError` (422).
- `app/application/use_cases/upload_evidence.py`: admite evidencia en `requiere_revision`.
- `app/application/use_cases/completar_evaluacion.py`: 409 `REVISION_SIN_EVIDENCIA`.

- `app/application/ai/prompts/validar_evidencia.py` : regla 8, versión **1.2.0** (queda registrada en `llamadas_llm.prompt_version`).
- `app/api/routes/respuestas.py` : `PATCH /revision` solo con estados resolutivos.
- `app/core/config.py` y `.env.example` : `JUSTIFICACION_MIN_CARACTERES=20` .
- Pruebas adaptadas a la decisión: `test_seguridad_integracion.py` (`test_usuario_no_puede_declarar_requiere_revision` pasa a `test_usuario_puede_declarar_requiere_revision_con_justificacion` , más una de inyección en ese estado), `test_bordes_na_y_evidencia.py` (`requiere_revision` admite evidencia) y `test_e2e_capa_ia.py` (versión del prompt 1.2.0).
- `../README.md` : tabla de estados (sección 5.3), endpoints, variable nueva y tabla de pruebas.

Sin migraciones: la columna ya admitía `requiere_revision` (VARCHAR 25).

Verificación

En el entorno de desarrollo de esta entrega (sin spaCy, por eso las pruebas de NER se omiten): **336 aprobadas, 8 omitidas**. Contra PostgreSQL real (uvicorn + Redis):

- Los 5 estados → 201 y quedan en la BD.
- Parcial o revisión con justificación corta → 422 `JUSTIFICACION_OBLIGATORIA` .
- Evidencia en `requiere_revision` → 202.
- Revisión humana a `requiere_revision` → 400.
- Latencia: 12/12 endpoints bajo 500 ms.

```
docker compose up -d --build api worker
docker compose exec worker pytest tests/test_hu02_matriz_estados.py -v
docker compose exec worker pytest tests -v -rxX
```

3. Corrección: la brecha sigue al estado vigente

Defecto detectado en la revisión posterior: la brecha se asignaba cuando el estado final pasaba a `no_cumplido` , pero **nunca se quitaba**. Por ejemplo, si el LLM decidía `no_cumplido` y el Oficial lo revisaba como `cumplido` , el control quedaba cumplido con brecha. Lo mismo pasaba si una evidencia nueva mejoraba el estado.

- Nueva regla única en el modelo: `Respuesta.sincronizar_brecha(control)` . La brecha del control se asigna si y solo si `estado_clasificacion_final = no_cumplido` .
- Se aplica en todos los puntos donde cambia el estado vigente:
  - declaración del usuario (`create_respuesta.py`);
  - revisión humana (`PATCH /revision`);
  - decisión del LLM, fallback por baja confianza y bloqueos de seguridad (`process_evidence.py`).
- **Migración** `d4a7c2e9b8f3` (solo datos, después de `b3c8e5d2a7f1`): quita la brecha de las filas existentes cuyo estado final no es `no_cumplido` . Probada en PostgreSQL 16 (upgrade,

downgrade y upgrade). El downgrade no restaura esas brechas, porque eran incorrectas.
- 6 pruebas nuevas en `test_hu02_matriz_estados.py` (15 casos). Antes de la corrección fallaban 14 de ellos.

Archivos: `app/domain/models/respuesta.py`, `app/application/use_cases/create_respuesta.py`, `app/application/use_cases/process_evidence.py`, `app/api/routes/respuestas.py`, `migrations/versions/d4a7c2e9b8f3_hu02_brecha_coherente.py`, `tests/test_hu02_matriz_estados.py`.

Pendiente de decisión del PO: `PATCH /revision` todavía permite cambiar la clasificación de una evaluación ya completada; el resumen y el benchmark no se recalculan. Opciones: bloquearlo con 409 `EVALUACION_CERRADA`, o permitirlo y recalcular el resumen (HU-22, bandeja de revisión).

Pendiente fuera del backend (Sprint 4, frontend)

El prototipo ya permite elegir "Requiere revisión" (`cuestionario.html`), pero:

- Le da 50 puntos (`logica.js`, `PESO_ESTADO.revision`), mientras que en el backend vale 0 hasta que se resuelva.
- No exige la justificación ni pide evidencia en ese estado.

El frontend definitivo debe consumir las reglas del backend: los 5 estados, el 422 `JUSTIFICACION_OBLIGATORIA` y el 409 `REVISION_SIN_EVIDENCIA`.